

- **Probabilidade**

1. Urna com bolas coloridas

Uma urna contém 5 bolas vermelhas e 7 azuis. Qual a probabilidade de retirar uma bola vermelha?

calcular a probabilidade de sair 2 vermelhas na sequência (com e sem reposição)

```
#===EX.1
```

```
t = 12
```

```
prob_vermelha = 5 / t
```

```
prob_com_repos = prob_vermelha * prob_vermelha
```

```
p_sem_repos = (5 - 1) / (t - 1)
```

```
prob_sem_repos = prob_vermelha * p_sem_repos
```

```
print(f"\nProbabilidade de tirar vermelha: {prob_vermelha:3f} ou {prob_vermelha:.2%} ")
```

```
print(f"Probabilidade de sair duas vermelhas na sequência COM reposição: {prob_com_repos:.3f}  
ou {prob_com_repos:.2%} ")
```

```
print(f"Probabilidade de sair duas vermelhas na sequência SEM reposição: {prob_sem_repos:.3f}  
ou {prob_sem_repos:.2%} ")
```

2. Lançamento de um dado

Qual a probabilidade de sair um número maior que 3 ao lançar um dado de 6 lados?

```
#===EX.2
```

```
d = 6
```

```
prob_dados = 3 / d
```

```
print(f"\nA probabilidade de sair um número maior que 3 ao lançar um dado de 6 lados é:  
{prob_dados} ou {prob_dados:.2%}")
```

3. Lançamento de duas moedas

Qual a probabilidade de sair exatamente uma cara ao lançar duas moedas?

```
####EX.3
```

```
m = 4
```

```
prob_moeda = 2 / m
```

```
print(f"\nA probabilidade de sair exatamente uma cara ao lançar duas moedas é: {prob_moeda}
```

```
ou ({prob_moeda:.2%})")
```

4. Probabilidade condicional

Uma urna tem 3 bolas vermelhas e 2 verdes. Se uma bola é retirada sem reposição e sai vermelha, qual a probabilidade da próxima ser verde?

```
####EX.4
```

```
prob_verde = 2 / (5 - 1)
```

```
print(f"\nProbabilidade da próxima ser verde dado que a primeira foi vermelha: {prob_verde:.2f}
```

```
({prob_verde:.2%})")
```

- **Estatística Descritiva com Panda**

5. Média, mediana e moda

Com o conjunto de dados: [10, 15, 20, 20, 25, 30, 35], calcule média, mediana e moda utilizando pandas

```
####EX.5
```

```
import pandas as pd
```

```
dados = pd.Series([10, 15, 20, 20, 25, 30, 35])
```

```
media = dados.mean()
```

```
mediana = dados.median()
```

```
moda = dados.mode()
```

```
print("Média:", media)
```

```
print("Mediana:", mediana)
print("Moda:", list(modos)) # Pode ter mais de uma moda
```

6. Variância e desvio padrão

Calcule a variância e o desvio padrão do conjunto: [2, 4, 4, 4, 5, 5, 7, 9], usando pandas.

```
#===EX.6
dados = pd.Series([2, 4, 4, 4, 5, 5, 7, 9])
variacao = dados.var()
desvio_padrao = dados.std()
print("Variância:", variacao)
print("Desvio padrão:", desvio_padrao)
```

7. Medidas resumo

Para os dados [5, 7, 8, 5, 10, 12, 15], calcule:

- o Média
- o Mediana
- o Valor mínimo
- o Valor máximo
- o Amplitude

```
#===EX.7
dados = pd.Series([5, 7, 8, 5, 10, 12, 15])

media = dados.mean()
mediana = dados.median()
minimo = dados.min()
maximo = dados.max()
amplitude = maximo - minimo

print("Média:", media)
print("Mediana:", mediana)
print("Valor mínimo:", minimo)
print("Valor máximo:", maximo)
print("Amplitude:", amplitude)
```

- **Funções Trigonométricas e Logarítmicas**

8. Enunciado:

Usando numpy, gere uma série de valores x (100 números entre 0.1 e 10) e calcule $y = \sin(x) + \log(x)$. Plote o gráfico.

```
#===EX.8

import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.1, 10, 100)
y = np.sin(x) + np.log(x)

plt.plot(x, y)
plt.title("y = sin(x) + log(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()
```

- **Regressão Linear**

9. Horas de estudo vs. notas

Considere os dados:

o Horas de estudo: [1, 2, 3, 4, 5]

o Notas: [2, 4, 5, 4, 6] Ajuste uma regressão linear simples e interprete o coeficiente angular.

```
#===EX.9

from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 6])

modelo = LinearRegression()
modelo.fit(X, y)
```

```
coef_angular = modelo.coef_[0]
```

```
intercepto = modelo.intercept_
```

```
print("Coeficiente angular (inclinação):", coef_angular)
```

```
print("Interpretação: A cada hora de estudo, a nota aumenta em média", round(coef_angular, 2))
```

10. Preço vs. tamanho de imóveis

Considere os dados:

o Tamanho (m²): [50, 60, 70, 80, 90]

o Preço (mil reais): [150, 200, 210, 240, 280] Ajuste um modelo de regressão linear e estime o preço para um imóvel de 100 m².

```
#===EX.10
```

```
X = np.array([50, 60, 70, 80, 90]).reshape(-1, 1)
```

```
y = np.array([150, 200, 210, 240, 280])
```

```
modelo = LinearRegression()
```

```
modelo.fit(X, y)
```

```
preco_estimado = modelo.predict([[100]])[0]
```

```
print("Coeficiente angular:", modelo.coef_[0])
```

```
print("Intercepto:", modelo.intercept_)
```

```
print("Preço estimado para 100 m2:", round(preco_estimado, 2), "mil reais")
```

• Visualizações com Matplotlib e Seaborn

11. Histograma (Matplotlib e Seaborn)

Gere 1000 números aleatórios com distribuição normal (média 60, desvio padrão 15).

o Plote um histograma com matplotlib.

o Plote o mesmo histograma com seaborn.

```
#===EX.11
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import numpy as np
```

```
np.random.seed(42)
```

```
dados = np.random.normal(loc=60, scale=15, size=1000)
```

```
# Matplotlib
```

```
plt.hist(dados, bins=30, edgecolor='black')
```

```
plt.title("Histograma com Matplotlib")
```

```
plt.xlabel("Valor")
```

```
plt.ylabel("Frequência")
```

```
plt.show()
```

```
# Seaborn
```

```
sns.histplot(dados, kde=True, bins=30)
```

```
plt.title("Histograma com Seaborn")
```

```
plt.xlabel("Valor")
```

```
plt.ylabel("Frequência")
```

```
plt.show()
```

12.Gráfico de dispersão (Seaborn)

Considere os dados:

o X = [1, 2, 3, 4, 5]

o Y = [2, 4, 5, 4, 6] Plote o gráfico de dispersão (scatter plot) usando seaborn.

```
#===EX.12
```

```
import seaborn as sns
```

```
import pandas as pd
```

```
dados = pd.DataFrame({
```

```
    'X': [1, 2, 3, 4, 5],
```

```
    'Y': [2, 4, 5, 4, 6]
```

```
})
```

```
sns.scatterplot(data=dados, x='X', y='Y')
plt.title("Gráfico de Dispersão com Seaborn")
plt.grid(True)
plt.show()
```

13.Boxplot (Seaborn)

Construa um boxplot para os dados: [7, 8, 5, 6, 12, 14, 15, 8, 9, 10] com seaborn.

```
#===EX.13
dados = [7, 8, 5, 6, 12, 14, 15, 8, 9, 10]
sns.boxplot(data=dados)
plt.title("Boxplot com Seaborn")
plt.xlabel("Valores")
plt.show()
```

14. CSV “industrial.csv”

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Carregando o CSV
data = pd.read_csv("UC4/projeto/industria.csv", parse_dates=["Data"])
# Visualizando os dados
print(data.head())
```

1. Receita total por fábrica

1. Calcule a **receita total** de cada fábrica.
2. Gere um **gráfico de barras** mostrando a receita total por fábrica.
3. Perguntas:
 - Qual fábrica teve a maior receita?
 - Qual a diferença entre a fábrica com maior receita e a de menor receita?

```
#===EX.1
```

```
# Receita total por fábrica
```

```
receita_por_fabrica = data.groupby("Fabrica")["Receita"].sum().sort_values(ascending=False)
```

```
# Gráfico de barras
```

```
sns.barplot(x=receita_por_fabrica.index, y=receita_por_fabrica.values)
```

```
plt.title("Receita Total por Fábrica")
```

```
plt.ylabel("Receita Total")
```

```
plt.xlabel("Fábrica")
```

```
plt.show()
```

```
# Perguntas
```

```
fabrica_maior_receita = receita_por_fabrica.idxmax()
```

```
fabrica_menor_receita = receita_por_fabrica.idxmin()
```

```
diferenca = receita_por_fabrica.max() - receita_por_fabrica.min()
```

```
print(f"Fábrica com maior receita: {fabrica_maior_receita}")
```

```
print(f"Diferença entre maior e menor receita: R${diferenca}")
```

2. Receita média por produto

1. Calcule a **receita média** de cada produto.
2. Gere um **gráfico de barras** mostrando a receita média por produto.
3. Perguntas:
 - Qual produto tem a maior receita média?
 - Algum produto apresenta receita média significativamente menor? Qual?

```
#===EX.2
```

```
# Receita média por produto
```

```
receita_media_produto =
```

```
data.groupby("Produto")["Receita"].mean().sort_values(ascending=False)
```



```
# Gráfico de barras
```

```
sns.barplot(x=receita_media_produto.index, y=receita_media_produto.values)
```

```
plt.title("Receita Média por Produto")
```

```
plt.ylabel("Receita Média")
```

```
plt.xlabel("Produto")
```

```
plt.show()
```

```
# Perguntas
```

```
produto_maior_receita_media = receita_media_produto.idxmax()
```

```
produto_menor_receita_media = receita_media_produto.idxmin()
```

```
print(f"Produto com maior receita média: {produto_maior_receita_media}")
```

```
print(f"Produto com menor receita média: {produto_menor_receita_media}")
```

3. Quantidade vendida total por mês

1. Crie uma coluna Mes a partir da data de cada registro.
2. Calcule a **quantidade total vendida** por mês.
3. Gere um **gráfico de linha** mostrando a evolução da quantidade vendida ao longo dos meses.
4. Perguntas:
 - Qual mês teve a maior quantidade vendida?
 - Existe tendência de aumento ou diminuição ao longo do período?

```
=====EX.3
```

```
# Criar coluna "Mes"
```

```
data["Mes"] = data["Data"].dt.to_period("M").astype(str)
```

```
# Quantidade total vendida por mês
```

```
quantidade_vendida_mes = data.groupby("Mes")["Quantidade_Vendida"].sum()
```

```
# Gráfico de linha
```

```
quantidade_vendida_mes.plot(kind="line", marker="o")
```

```
plt.title("Quantidade Vendida Total por Mês")
```

```
plt.ylabel("Quantidade Vendida")
plt.xlabel("Mês")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

```
# Perguntas
```

```
mes_maior_venda = quantidade_vendida_mes.idxmax()
print(f"Mês com maior quantidade vendida: {mes_maior_venda}")
```

4. Lucro médio por fábrica

1. Crie uma coluna Lucro = Receita - Custo.
2. Calcule o **lucro médio por fábrica** usando groupby.
3. Gere um **gráfico de barras** usando seaborn mostrando o lucro médio por fábrica.
4. Perguntas:
 - Qual fábrica é mais lucrativa em média?
 - Alguma fábrica apresenta lucro negativo em algum registro?

```
===EX.4
```

```
# Criar coluna "Lucro"
```

```
data["Lucro"] = data["Receita"] - data["Custo"]
```

```
# Lucro médio por fábrica
```

```
lucro_medio_fabrica = data.groupby("Fabrica")["Lucro"].mean().sort_values(ascending=False)
```

```
# Gráfico de barras
```

```
sns.barplot(x=lucro_medio_fabrica.index, y=lucro_medio_fabrica.values)
```

```
plt.title("Lucro Médio por Fábrica")
```

```
plt.ylabel("Lucro Médio")
```

```
plt.xlabel("Fábrica")
```

```
plt.show()
```

```
# Perguntas
```

```
fabrica_mais_lucrativa = lucro_medio_fabrica.idxmax()
```

```
lucros_negativos = data[data["Lucro"] < 0]
```

```
print(f"Fábrica mais lucrativa em média: {fabrica_mais_lucrativa}")
```

```
print(f"Existe algum lucro negativo? {'Sim' if not lucros_negativos.empty else 'Não'})
```

5. Receita total por fábrica e produto

1. Use groupby ou pivot_table para calcular a **receita total** para cada combinação de fábrica e produto.
2. Gere um **heatmap** com seaborn mostrando a receita por fábrica (linhas) e produto (colunas).
3. Perguntas:
 - Qual produto gera mais receita em cada fábrica?
 - Existe algum produto que não foi produzido ou vendido em alguma fábrica?

```
#===EX.5
```

```
# Tabela de receita total por fábrica e produto
```

```
pivot_receita = pd.pivot_table(data, values="Receita", index="Fabrica", columns="Produto",  
aggfunc="sum", fill_value=0)
```

```
# Heatmap
```

```
plt.figure(figsize=(8, 5))
```

```
sns.heatmap(pivot_receita, annot=True, fmt=".0f", cmap="YlGnBu")
```

```
plt.title("Receita Total por Fábrica e Produto")
```

```
plt.xlabel("Produto")
```

```
plt.ylabel("Fábrica")
```

```
plt.show()
```

```
# Perguntas
```

```
for fabrica in pivot_receita.index:
```

```
    produto_mais_receita = pivot_receita.loc[fabrica].idxmax()
```

```
    print(f"Produto que gera mais receita na {fabrica}: {produto_mais_receita}")
```

```
faltantes = pivot_receita == 0
```

```
if faltantes.any().any():
```

```
    print("\nProdutos que não foram vendidos em algumas fábricas.")
```

```
    print(pivot_receita.where(faltantes).dropna(how='all').dropna(axis=1, how='all'))
```

```
else:
```

```
    print("\nTodos os produtos foram vendidos em todas as fábricas.")
```